

power of having a database of hashes is not fully being exploited. An attack using such a database would consume too much memory, and a direct attack would be too time-consuming. The trade-off here is between too much time (days, months, years), or too much space (TB, PB, XB). If only there was a middle ground, which used lesser space than a fully exhaustive database of hashes for all possible strings, and one which would take significantly lesser time than a brute-force attack.

This is where rainbow tables come in. Before going in to rainbow tables, let us first understand their predecessors, hash chains.

The concept of hash chains allows for a good compromise between the speed of using a hash database and the memory usage of bruteforcing. Hash chains allow for saving significant amounts of space by finding out the hashes for limited number of texts and deriving the rest from them.

Hash chains rely on two functions, the hashing function and the reduction function. While the hash function – as expected – maps a plaintext to a hash, the reduction function maps a hash to a plaintext.

Here when we say that the reduction function maps a hash to a plaintext, it does not mean that it is finding the original plaintext that created the hash, but a plaintext that is derived from the hash. For those who remember sets in mathematics, a hash function is a function that maps the set of all plaintexts to a set of all hashes; a reduction function is a function that maps the set of all hashes to a set of all plaintexts. The plaintext that the reduce function will return will not be the same one which was used to create the hash; however it will be a valid plaintext nonetheless.

**Example:**

**The mathematical function:**

$$h(x) = 2x + 1$$

Operates on a set of all integers, and maps the set of integers to a set of odd numbers.

**Another function:**

$$r(x) = \frac{x+3}{2}$$

Operates on a set of all odd numbers, and maps the set of odd number to a set of integers.

Here, the function  $h(x)$  will always map to an odd number (comparable to the “hash”), and given this hash, the function  $r(x)$  function will always result in an integer (comparable to the “plaintext”). However, it is important to note that the reduction of a hash will not result in the original plaintext.